

KDMS SW 아키텍처

KDMS-D02 · v0.1 · 2026-10-06

Maven 단일 모듈 · 패키지 계층 · 위 계층만 아래 계층을 쓴다

의
주
방
향

진입 계층

같은 기능을
두 입구로



Kdms (main)

단일 jar 시작점
--no-web 이면 CLI 만



kdms.cli

picocli 명령 · status · init · web
plan · load · sync · cutover · verify



kdms.web

Spring MVC · Thymeleaf · 상태 REST
127.0.0.1 에만 바인딩

이관 엔진

단계마다 채운다
(아래 줄 = 구현 단계)



kdms.ddl

대상 DDL 생성·적용
2단계



kdms.load

구간 분할 · COPY
3단계



kdms.verify

건수 · 합계 · 해시
3단계



kdms.cdc

Debezium Embedded
4단계



kdms.apply

LSN 순서 먹등 반영
4단계



kdms.cutover

전환 상태 기계
5단계

공통 계층

모든 엔진이
함께 쓴다



kdms.catalog

원천·대상 메타데이터
접속 확인 1단계 · 카탈로그 2단계



kdms.rules

kdms-rules.yml 읽기·검사
타입·값 변환기 선택



kdms.state

관리 테이블 · 작업 상태 기계
pg_advisory_lock 동시 실행 방지



kdms.config

YAML 설정 · .env 치환
JDBC 연결

외부 라이브러리

모두 jar 에 포함
실행 중 다운로드 0



Spring Boot 4.1

내장 Tomcat · MVC
Thymeleaf



Debezium 3.7

Embedded · SQL Server
JDBC 오프셋 저장소



JDBC 드라이버

PgJDBC
mssql-jdbc



picocli 4.7

명령줄 해석



SnakeYAML · Logback

설정 읽기 · 로그

범례 → 실시간 질의 --> 비동기 데이터 흐름

점선: CDC 가 대상 PG 의 kdms.change_log 에 쌓고 반영기가 따로 읽는다(수집과 반영 분리)

강조: 무중단의 핵심(CDC 수집·반영) · 엔진 사이 순서는 kdms.state 의 작업 상태로 정한다

1. 계층과 패키지

계층	패키지	책임	구현 단계
진입	<code>kdms</code> (<code>Kdms.java</code>)	jar 시작점. 명령을 해석해 CLI 로 실행하거나 내장 웹을 띄운다	1단계
진입	<code>kdms.cli</code>	picocli 명령. 지금 <code>status</code> · <code>init</code> · <code>web</code> , 단계마다 <code>plan</code> · <code>load</code> · <code>sync</code> · <code>cutover</code> · <code>verify</code> 추가	1단계~
진입	<code>kdms.web</code>	Thymeleaf 화면 + 직접 작성한 JS·CSS(외부 CDN 없음), 상태 REST(Representational State Transfer, 상태 조회·조작 API). 기본 <code>127.0.0.1</code>	1단계~
엔진	<code>kdms.ddl</code>	대상 DDL 생성·적용: 테이블, PK, UNIQUE, 시퀀스, 나중에 쉼 FK	2단계
엔진	<code>kdms.load</code>	전체 적재: 구간 분할, SNAPSHOT 격리로 읽기, PgJDBC <code>CopyManager</code> 로 COPY, 구간 상태 기록	3단계
엔진	<code>kdms.verify</code>	건수·합계·해시 SQL 생성(원천 T-SQL / 대상 SQL), 비교, 행 단위 차이	3단계
엔진	<code>kdms.cdc</code>	Debezium Embedded 엔진 수명주기, 이벤트 → <code>kdms.change_log</code> 저장	4단계
엔진	<code>kdms.apply</code>	<code>change_log</code> → 대상 테이블 먹통 적용, 반영 위치 기록	4단계
엔진	<code>kdms.cutover</code>	전환 상태 기계: 캡처 지연 확인 → 마지막 반영 0건 2회 → 검증 → <code>setval</code> → FK	5단계
공통	<code>kdms.catalog</code>	원천·대상 메타데이터. 1단계는 접속·준비 상태(<code>status</code>), 2단계부터 <code>sys.*</code> 카탈로그	1·2단계
공통	<code>kdms.rules</code>	변환 규칙 파일 읽기·검사, 원천 컬럼 → 대상 타입·값 변환기 선택	1·2단계
공통	<code>kdms.state</code>	관리 테이블(<code>kdms</code> 스키마) 생성·접근, 작업 상태 기계, <code>pg_advisory_lock</code>	1단계~
공통	<code>kdms.config</code>	YAML 설정 읽기·검사, <code>.env</code> 환경 변수 치환, JDBC 연결	1단계

2. 규칙

- 의존은 위에서 아래로만: 진입 → 엔진 → 공통 → 라이브러리. 엔진 사이 순서는 `kdms.state` 의 작업 상태로 정한다(예: 전체 적재가 끝난 테이블부터 반영기가 시작, plan §4.4). 전환 (`kdms.cutover`)은 반영·검증을 차례로 부른다(plan §4.5).
- Maven 단일 모듈, 패키지로만 나눈다(plan §4.1). 외부 라이브러리는 모두 jar 에 넣고, 라이선스는 빌드 때 자동 검사한다(licenses.md §5).
- Debezium 은 포크하지 않고 Maven 의존성과 공개 인터페이스로만 쓴다. Kafka Connect REST 서버용 Jetty-Jersey 등은 뺐다(licenses.md §5.2).
- 화면은 외부 프론트엔드 라이브러리 없이 서버 렌더링. 웹과 CLI 는 같은 엔진을 부른다.